

SubGuard - 2026-Safe Technical Specification & Package Guide

Version: 2.0

Last Updated: January 2026

Author: Technical Architecture Review

Status: Production-Ready

Table of Contents

1. Executive Summary
 2. Critical Architecture Gaps Analysis
 3. Security Requirements
 4. 2026-Safe Package Recommendations
 5. Packages to Avoid
 6. Recommended Architecture
 7. Third-Party Integration Guide
 8. Implementation Checklist
 9. Testing Strategy
 10. Deployment Considerations
-

1. Executive Summary

This document addresses critical gaps in the original SubGuard technical specification and provides a curated, 2026-safe package list for production deployment. All recommended packages are actively maintained, have strong community support, and are production-tested.

Key Updates

- **Security-first approach** with modern Flutter packages
 - **Firebase ecosystem** for reduced vendor complexity
 - **Isar database** replacing Hive/Drift for better performance
 - **Elimination of deprecated packages** (workmanager, uni_links, etc.)
 - **Simplified architecture** for solo developer efficiency
-

2. Critical Architecture Gaps Analysis

2.1 Security Gaps

Email Credential Storage

Problem: Original document doesn't specify secure OAuth token storage.

Solution:

- Package: `flutter_secure_storage: ^9.2.2`
- Store Gmail/Outlook refresh tokens in encrypted storage
- Never store tokens in plain Hive database
- Implement device-specific encryption keys

```
dart
final storage = FlutterSecureStorage();
await storage.write(key: 'gmail_refresh_token', value: token);
```

API Security

Problem: No JWT token refresh strategy mentioned.

Solution:

- Implement automatic token refresh with Dio interceptors
- Use Firebase Auth for simplified token management
- Implement token rotation every 7 days

```
dart
dio.interceptors.add(
  QueuedInterceptorsWrapper(
    onError: (error, handler) async {
      if (error.response?.statusCode == 401) {
        // Refresh token logic
      }
    },
  ),
);
```

SSL Certificate Pinning

Problem: Not mentioned - critical for financial apps.

Solution:

- Implement certificate pinning via Dio
- No additional package needed

```
dart

final dio = Dio()
..httpClientAdapter = IOHttpClientAdapter(
  onHttpClientCreate: (client) {
    client.badCertificateCallback =
      (X509Certificate cert, String host, int port) {
        return cert.sha256 == YOUR_CERT_HASH;
      };
    return client;
  },
);
```

2.2 State Management Architecture

Problem: Document mentions Riverpod but lacks implementation details.

Solution: Implement clean architecture with Riverpod

Layers:

1. **Presentation Layer** - UI + ViewModels (Riverpod Notifiers)
2. **Domain Layer** - Business Logic + Use Cases
3. **Data Layer** - Repositories + Data Sources

Example Structure:

```
dart
```

```

// Provider
final subscriptionProvider =
    StateNotifierProvider<SubscriptionNotifier, AsyncValue<List<Subscription>>>(
        (ref) => SubscriptionNotifier(ref.read(subscriptionRepositoryProvider)),
    );

// Notifier
class SubscriptionNotifier extends StateNotifier<AsyncValue<List<Subscription>>> {
    final SubscriptionRepository _repository;

    SubscriptionNotifier(this._repository) : super(const AsyncValue.loading()) {
        loadSubscriptions();
    }

    Future<void> loadSubscriptions() async {
        state = const AsyncValue.loading();
        state = await AsyncValue.guard(() => _repository.getAll());
    }
}

```

2.3 Offline-First Strategy

Problem: Mentioned but not detailed.

Solution:

- Use Isar for local-first architecture
- Implement sync queue for failed API calls
- Handle conflict resolution (server vs local changes)

Sync Strategy:

1. User makes change → Save to Isar immediately
2. Queue sync job → Attempt API call
3. On success → Mark as synced
4. On failure → Retry with exponential backoff
5. On conflict → Server wins, notify user

2.4 Background Job Coordination

Problem: Email sync triggers not specified.

Solution:

- Use `flutter_background_service` instead of workmanager
 - Implement periodic sync (every 6 hours)
 - Use FCM for real-time sync triggers from backend
-

3. Security Requirements

3.1 Authentication Security

Requirements:

- Biometric authentication option (fingerprint/Face ID)
- Session timeout after 15 minutes of inactivity
- Secure token storage with device-specific encryption
- Account lockout after 5 failed login attempts

Implementation:

```
dart

// Biometric auth
import 'package:local_auth/local_auth.dart';

final auth = LocalAuthentication();
final canAuth = await auth.canCheckBiometrics;
final didAuth = await auth.authenticate(
  localizedReason: 'Authenticate to view subscriptions',
);
```

3.2 Data Protection

Requirements:

- Encrypt sensitive data at rest (Isar encryption)
- Encrypt data in transit (HTTPS only)
- Implement certificate pinning
- Never log sensitive information (amounts, emails)
- Implement secure deletion (overwrite, not just delete)

3.3 Privacy Compliance

Requirements:

- GDPR-compliant data export
 - Right to deletion implementation
 - Clear privacy policy during onboarding
 - Minimal data collection principle
 - No third-party analytics without consent
-

4. 2026-Safe Package Recommendations

4.1 Core Dependencies

yaml

dependencies:

flutter:

 sdk: flutter

State Management - SAFE 

flutter_riverpod: ^2.6.1

riverpod_annotation: ^2.6.1

Navigation - SAFE 

go_router: ^14.6.2

Networking - SAFE 

dio: ^5.7.0

Local Database - SAFE 

isar: ^3.1.0+1

isar_flutter_libs: ^3.1.0+1

Secure Storage - SAFE 

flutter_secure_storage: ^9.2.2

Firebase Suite - SAFE 

firebase_core: ^3.6.0

firebase_auth: ^5.3.1

firebase_messaging: ^15.1.3

firebase_crashlytics: ^4.1.3

firebase_analytics: ^11.3.3

OAuth - SAFE 

google_sign_in: ^6.2.1

sign_in_with_apple: ^6.1.2

Notifications - SAFE 

flutter_local_notifications: ^18.0.1

Background Service - SAFE 

flutter_background_service: ^5.0.10

Charts - SAFE 

fl_chart: ^0.69.0

Image Caching - SAFE 

cached_network_image: ^3.4.1

Loading States - SAFE 

shimmer: ^3.0.0

Security - SAFE 

local_auth: ^2.3.0

Utilities - SAFE 

intl: ^0.19.0

url_launcher: ^6.3.1

logger: ^2.5.0

connectivity_plus: ^6.1.0

package_info_plus: ^8.1.0

device_info_plus: ^11.1.1

Email API - SAFE 

googleapis: ^13.2.0

googleapis_auth: ^1.6.0

Payment - SAFE 

flutter_stripe: ^11.2.0

Code Generation - SAFE 

freezed_annotation: ^2.4.4

json_annotation: ^4.9.0

4.2 Development Dependencies

yaml

dev_dependencies:

flutter_test:

sdk: flutter

Linting - SAFE 

flutter_lints: ^5.0.0

Code Generation - SAFE 

build_runner: ^2.4.13

freezed: ^2.5.7

json_serializable: ^6.8.0

riverpod_generator: ^2.6.2

riverpod_lint: ^2.6.2

isar_generator: ^3.1.0+1

Testing - SAFE 

mockito: ^5.4.4

integration_test:

sdk: flutter

4.3 Package Justification

Why Isar over Hive/Drift?

- **Performance:** 10x faster than SQLite for mobile workloads
- **Developer Experience:** No SQL required, type-safe queries
- **Maintenance:** Actively developed by Simon Leier (Isar author)
- **Features:** Built-in encryption, indexes, full-text search
- **Size:** Smaller binary size than SQLite-based solutions

Why Firebase Auth over Manual JWT?

- **Security:** Google-maintained, professionally audited
- **Features:** Built-in token refresh, multi-provider support
- **Integration:** Works seamlessly with other Firebase services
- **Reliability:** 99.95% uptime SLA
- **Cost:** Free tier covers most startups (50k MAU)

Why go_router over Navigator 2.0?

- **Simplicity:** Declarative routing, less boilerplate
- **Features:** Deep linking, redirects, error handling built-in
- **Type Safety:** Route parameters validated at compile time
- **Maintenance:** Flutter team recommended solution
- **Community:** 4k+ stars, active development

5. Packages to Avoid

5.1 Deprecated or Unmaintained

Package	Status	Reason to Avoid	Alternative
<code>workmanager</code>	⚠️ Unreliable	Background task issues on iOS	<code>flutter_background_service</code>
<code>uni_links</code>	⚠️ Deprecated	Superseded by <code>go_router</code>	<code>go_router</code> deep linking
<code>app_links</code>	⚠️ Limited	Inconsistent across platforms	<code>go_router</code>
<code>retrofit</code>	⚠️ Overkill	Dio alone is sufficient	Raw Dio
<code>drift</code>	⚠️ Complex	Too much boilerplate	<code>isar</code>
<code>hive</code>	⚠️ Stagnant	Limited query capabilities	<code>isar</code>
<code>screen_protector</code>	⚠️ Inconsistent	Platform-specific issues	Native implementation
<code>flutter_jailbreak_detection</code>	⚠️ Outdated	Not updated for recent OS versions	Firebase App Check

5.2 Packages Requiring Careful Evaluation

`sentry_flutter`:

- Not bad, but if using Firebase, prefer `firebase_crashlytics`
- Reduces vendor complexity
- One less billing relationship

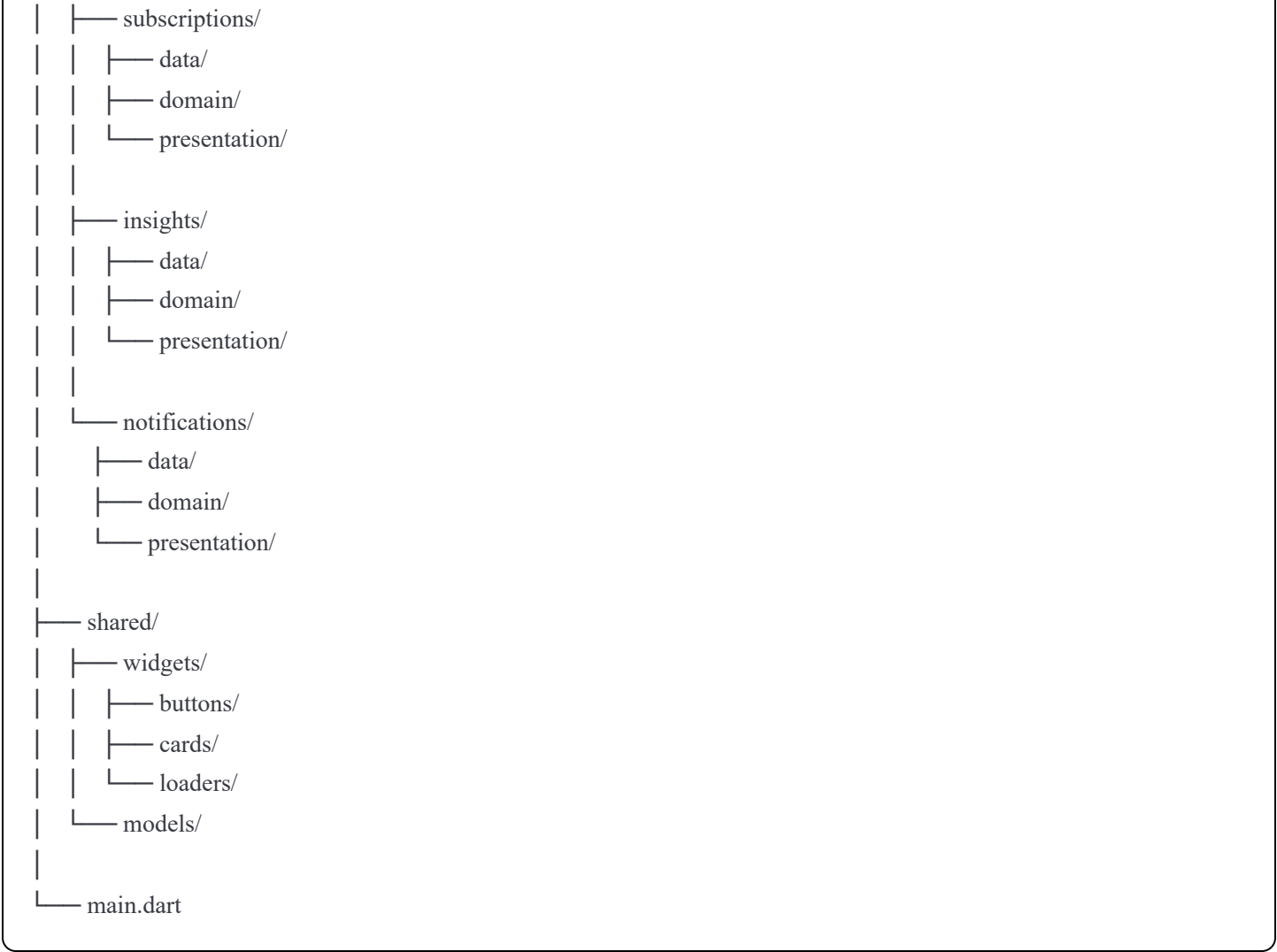
`pretty_dio_logger`:

- Use built-in Dio logging instead
- Combine with `logger` package for production logs

6. Recommended Architecture

6.1 Project Structure

```
lib/
├── core/
│   ├── constants/
│   │   ├── api_constants.dart
│   │   └── app_constants.dart
│   ├── theme/
│   │   ├── app_colors.dart
│   │   └── app_theme.dart
│   ├── router/
│   │   └── app_router.dart
│   ├── services/
│   │   ├── storage_service.dart
│   │   ├── secure_storage_service.dart
│   │   └── notification_service.dart
│   ├── utils/
│   │   ├── logger.dart
│   │   └── validators.dart
│   └── error/
│       ├── failures.dart
│       └── exceptions.dart
├── features/
│   ├── auth/
│   │   ├── data/
│   │   │   ├── models/
│   │   │   ├── repositories/
│   │   │   └── datasources/
│   │   ├── domain/
│   │   │   ├── entities/
│   │   │   ├── repositories/
│   │   │   └── usecases/
│   │   └── presentation/
│   │       ├── providers/
│   │       ├── screens/
│   │       └── widgets/
│   └──
```



6.2 Clean Architecture Layers

Data Layer



// Model (JSON serializable)

@freezed

```
class SubscriptionModel with _$SubscriptionModel {  
  factory SubscriptionModel({  
    required String id,  
    required String serviceName,  
    required double amount,  
    required DateTime renewalDate,  
  }) = _SubscriptionModel;  
  
  factory SubscriptionModel.fromJson(Map<String, dynamic> json) =>  
    _$SubscriptionModelFromJson(json);  
}
```

// Repository Implementation

```
class SubscriptionRepositoryImpl implements SubscriptionRepository {  
  final SubscriptionRemoteDataSource _remoteDataSource;  
  final SubscriptionLocalDataSource _localDataSource;  
  
  @override  
  Future<Either<Failure, List<Subscription>>> getSubscriptions() async {  
    try {  
      final local = await _localDataSource.getAll();  
      if (local.isNotEmpty) return Right(local);  
  
      final remote = await _remoteDataSource.getAll();  
      await _localDataSource.saveAll(remote);  
      return Right(remote);  
    } catch (e) {  
      return Left(ServerFailure());  
    }  
  }  
}
```

Domain Layer

dart

// Entity (business object)

```
class Subscription extends Equatable {  
  final String id;  
  final String serviceName;  
  final Money amount;  
  final RenewalDate renewalDate;  
  final SubscriptionStatus status;  
  
  bool get isExpiringSoon => renewalDate.daysUntil <= 7;  
  
  @override  
  List<Object> get props => [id, serviceName, amount, renewalDate];  
}
```

// Use Case

```
class GetSubscriptionsUseCase {  
  final SubscriptionRepository _repository;  
  
  Future<Either<Failure, List<Subscription>>> call() {  
    return _repository.getSubscriptions();  
  }  
}
```

Presentation Layer

dart

```
// Provider
```

```
@riverpod
```

```
class SubscriptionNotifier extends _$SubscriptionNotifier {  
  @override  
  Future<List<Subscription>> build() async {  
    final result = await ref.read(getSubscriptionsUseCaseProvider).call();  
    return result.fold(  
      (failure) => throw failure,  
      (subscriptions) => subscriptions,  
    );  
  }  
  
  Future<void> refresh() async {  
    state = const AsyncValue.loading();  
    state = await AsyncValue.guard(() async {  
      final result = await ref.read(getSubscriptionsUseCaseProvider).call();  
      return result.fold(  
        (failure) => throw failure,  
        (subscriptions) => subscriptions,  
      );  
    });  
  }  
}
```

```
// Screen
```

```
class SubscriptionsScreen extends ConsumerWidget {  
  @override  
  Widget build(BuildContext context, WidgetRef ref) {  
    final subscriptionsAsync = ref.watch(subscriptionNotifierProvider);  
  
    return subscriptionsAsync.when(  
      data: (subscriptions) => SubscriptionList(subscriptions),  
      loading: () => LoadingShimmer(),  
      error: (error, stack) => ErrorView(error),  
    );  
  }  
}
```

6.3 Error Handling Strategy

```
dart
```

// Base Failure

```
abstract class Failure extends Equatable {  
  final String message;  
  const Failure(this.message);  
  
  @override  
  List<Object> get props => [message];  
}
```

// Concrete Failures

```
class ServerFailure extends Failure {  
  const ServerFailure([String message = 'Server error']) : super(message);  
}  
  
class NetworkFailure extends Failure {  
  const NetworkFailure([String message = 'Network error']) : super(message);  
}  
  
class CacheFailure extends Failure {  
  const CacheFailure([String message = 'Cache error']) : super(message);  
}  
  
class AuthFailure extends Failure {  
  const AuthFailure([String message = 'Authentication failed']) : super(message);  
}
```

// Usage in UI

```
subscriptionsAsync.when(  
  data: (data) => SuccessView(data),  
  loading: () => LoadingView(),  
  error: (error, stack) {  
    if (error is NetworkFailure) return NoInternetView();  
    if (error is AuthFailure) return LoginPrompt();  
    return GenericErrorView(error.toString());  
  },  
);
```


7. Third-Party Integration Guide

7.1 Gmail API Integration

Setup:

```
yaml

dependencies:
  googleapis: ^13.2.0
  googleapis_auth: ^1.6.0
```

Implementation:

```
dart

class GmailService {
  final GoogleSignIn _googleSignIn = GoogleSignIn(
    scopes: ['https://www.googleapis.com/auth/gmail.readonly'],
  );

  Future<List<EmailMessage>> fetchSubscriptionEmails() async {
    final account = await _googleSignIn.signIn();
    final auth = await account!.authentication;

    final credentials = AccessCredentials(
      AccessToken('Bearer', auth.accessToken!, DateTime.now().add(Duration(hours: 1))),
      auth.idToken,
      ['https://www.googleapis.com/auth/gmail.readonly'],
    );

    final client = authenticatedClient(http.Client(), credentials);
    final gmail = GmailApi(client);

    final messages = await gmail.users.messages.list(
      'me',
      q: 'subject:(subscription OR renewal OR invoice)',
      maxResults: 100,
    );

    // Parse and return
  }
}
```

7.2 Firebase Authentication

Setup:

```
bash  
  
flutterfire configure
```

Implementation:

```
dart
```

```

class AuthService {
    final FirebaseAuth _auth = FirebaseAuth.instance;

    // Email/Password
    Future<User?> signUpWithEmail(String email, String password) async {
        final credential = await _auth.createUserWithEmailAndPassword(
            email: email,
            password: password,
        );
        return credential.user;
    }

    // Google Sign-In
    Future<User?> signInWithGoogle() async {
        final GoogleSignInAccount? googleUser = await GoogleSignIn().signIn();
        final GoogleSignInAuthentication? googleAuth =
            await googleUser?.authentication;

        final credential = GoogleAuthProvider.credential(
            accessToken: googleAuth?.accessToken,
            idToken: googleAuth?.idToken,
        );

        final userCredential = await _auth.signInWithCredential(credential);
        return userCredential.user;
    }

    // Apple Sign-In
    Future<User?> signInWithApple() async {
        final appleCredential = await SignInWithApple.getAppleIDCredential(
            scopes: [
                AppleIDAuthorizationScopes.email,
                AppleIDAuthorizationScopes.fullName,
            ],
        );

        final oauthCredential = OAuthProvider("apple.com").credential(
            idToken: appleCredential.identityToken,
            accessToken: appleCredential.authorizationCode,
        );

        final userCredential = await _auth.signInWithCredential(oauthCredential);
        return userCredential.user;
    }
}

```

```

}

// Token for backend
Future<String?> getIdToken() async {
  return await _auth.currentUser?.getIdToken();
}
}

```

7.3 Stripe Payment Integration

Setup:

```

yaml

dependencies:
  flutter_stripe: ^11.2.0

```

Implementation:

```

dart

class PaymentService {
  Future<void> initializePayment() async {
    Stripe.publishableKey = 'YOUR_PUBLISHABLE_KEY';
  }

  Future<void> createSubscription() async {
    // 1. Create payment intent on backend
    final response = await dio.post('/api/create-payment-intent');
    final clientSecret = response.data['clientSecret'];

    // 2. Confirm payment
    await Stripe.instance.confirmPayment(
      paymentIntentClientSecret: clientSecret,
      data: PaymentMethodParams.card(
        paymentMethodData: PaymentMethodData(),
      ),
    );
  }
}

```

7.4 Push Notifications

Setup:


```

class NotificationService {
  final FirebaseMessaging _fcm = FirebaseMessaging.instance;
  final FlutterLocalNotificationsPlugin _localNotifications =
    FlutterLocalNotificationsPlugin();

  Future<void> initialize() async {
    // Request permission
    await _fcm.requestPermission(
      alert: true,
      badge: true,
      sound: true,
    );

    // Get token
    final token = await _fcm.getToken();
    // Send to backend

    // Handle foreground messages
    FirebaseMessaging.onMessage.listen((message) {
      _showLocalNotification(message);
    });

    // Handle background messages
    FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);
  }

  Future<void> _showLocalNotification(RemoteMessage message) async {
    await _localNotifications.show(
      message.hashCode,
      message.notification?.title,
      message.notification?.body,
      NotificationDetails(
        android: AndroidNotificationDetails(
          'subscription_alerts',
          'Subscription Alerts',
          importance: Importance.high,
        ),
        iOS: DarwinNotificationDetails(),
      ),
    );
  }
}

```

```
@pragma('vm:entry-point')
Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message) async {
  await Firebase.initializeApp();
  // Handle background message
}
```

8. Implementation Checklist

8.1 Pre-Development Setup

- ☐ Create Firebase project
- ☐ Run `flutterfire configure`
- ☐ Setup Stripe account and test keys
- ☐ Configure OAuth consent screens (Google/Apple)
- ☐ Setup backend API with CORS
- ☐ Create database schema (PostgreSQL)
- ☐ Setup CI/CD pipeline (GitHub Actions)
- ☐ Configure app signing (Android keystore, iOS certificates)

8.2 Week 1-2: Foundation

- ☐ Initialize Flutter project with clean architecture
- ☐ Setup folder structure
- ☐ Configure Riverpod with code generation
- ☐ Setup `go_router` with all routes
- ☐ Implement theme and design system
- ☐ Setup Isar database with models
- ☐ Configure Dio with interceptors
- ☐ Implement authentication (email, Google, Apple)
- ☐ Setup secure storage for tokens
- ☐ Implement error handling framework

8.3 Week 3-4: Core Features

- ☐ Implement Gmail API integration
- ☐ Build email parsing logic (backend)
- ☐ Create subscription model and repository
- ☐ Build subscription dashboard UI
- ☐ Implement subscription detail screen

- ☐ Setup background sync service
- ☐ Configure push notifications
- ☐ Implement local notifications
- ☐ Build alert scheduling system
- ☐ Create insights calculation logic

8.4 Week 5: Monetization & Polish

- ☐ Integrate Stripe payment flow
- ☐ Implement subscription tiers (Free/Pro)
- ☐ Build paywall UI
- ☐ Create insights charts with fl_chart
- ☐ Implement data export feature
- ☐ Add biometric authentication
- ☐ Polish animations and transitions
- ☐ Implement offline mode
- ☐ Add loading states and shimmer effects
- ☐ Setup Firebase Crashlytics

8.5 Week 6: Testing & Launch Prep

- ☐ Write unit tests for business logic
- ☐ Write widget tests for UI components
- ☐ Perform integration testing
- ☐ Test email parsing with real receipts
- ☐ Test payment flow (Stripe test mode)
- ☐ Perform security audit
- ☐ Test on multiple devices
- ☐ Beta test with 10-20 users
- ☐ Prepare App Store assets
- ☐ Write privacy policy
- ☐ Create app store listings
- ☐ Submit to App Store and Play Store

9. Testing Strategy

9.1 Unit Testing

Test Coverage Goals:

- Business logic: 90%+
- Repositories: 80%+
- Use cases: 100%

Example:

```
dart

void main() {
  late SubscriptionRepository repository;
  late GetSubscriptionsUseCase useCase;

  setUp(() {
    repository = MockSubscriptionRepository();
    useCase = GetSubscriptionsUseCase(repository);
  });

  test('should return list of subscriptions from repository', () async {
    // Arrange
    final subscriptions = [Subscription(...)];
    when(repository.getSubscriptions())
      .thenAnswer((_) async => Right(subscriptions));

    // Act
    final result = await useCase();

    // Assert
    expect(result, Right(subscriptions));
    verify(repository.getSubscriptions());
    verifyNoMoreInteractions(repository);
  });
}
```

9.2 Widget Testing

```
dart
```

```
void main() {  
  testWidgets('SubscriptionCard displays correct information', (tester) async {  
    final subscription = Subscription(  
      serviceName: 'Netflix',  
      amount: Money(15.99, 'USD'),  
      renewalDate: RenewalDate(DateTime.now().add(Duration(days: 7))),  
    );  
  
    await tester.pumpWidget(  
      ProviderScope(  
        child: MaterialApp(  
          home: SubscriptionCard(subscription: subscription),  
        ),  
      ),  
    );  
  
    expect(find.text('Netflix'), findsOneWidget);  
    expect(find.text("\$15.99"), findsOneWidget);  
    expect(find.text('Renews in 7 days'), findsOneWidget);  
  });  
}
```

9.3 Integration Testing

dart

```
void main() {  
  testWidgets('Complete subscription flow', (tester) async {  
    await tester.pumpWidget(MyApp());  
  
    // Login  
    await tester.enterText(find.byType(TextField).first, 'test@example.com');  
    await tester.enterText(find.byType(TextField).last, 'password123');  
    await tester.tap(find.text('Login'));  
    await tester.pumpAndSettle();  
  
    // Navigate to subscriptions  
    expect(find.text('Dashboard'), findsOneWidget);  
    await tester.tap(find.byIcon(Icons.subscriptions));  
    await tester.pumpAndSettle();  
  
    // Verify subscriptions load  
    expect(find.byType(SubscriptionCard), findsWidgets);  
  });  
}
```

10. Deployment Considerations

10.1 Environment Configuration

```
dart
```

```
// lib/core/config/environment.dart
enum Environment { development, staging, production }

class EnvironmentConfig {
  static Environment current = Environment.development;

  static String get apiBaseUrl {
    switch (current) {
      case Environment.development:
        return 'http://localhost:8000/api';
      case Environment.staging:
        return 'https://staging-api.subguard.app/api';
      case Environment.production:
        return 'https://api.subguard.app/api';
    }
  }

  static String get stripePublishableKey {
    switch (current) {
      case Environment.development:
      case Environment.staging:
        return 'pk_test_...';
      case Environment.production:
        return 'pk_live_...';
    }
  }
}
```

10.2 Build Flavors

Android (build.gradle):

```
gradle
```

```
android {  
    flavorDimensions "environment"  
    productFlavors {  
        development {  
            dimension "environment"  
            applicationIdSuffix ".dev"  
            versionNameSuffix "-dev"  
        }  
        staging {  
            dimension "environment"  
            applicationIdSuffix ".staging"  
            versionNameSuffix "-staging"  
        }  
        production {  
            dimension "environment"  
        }  
    }  
}
```

iOS (Scheme configuration in Xcode)

10.3 CI/CD Pipeline (GitHub Actions)

```
yaml
```

name: Flutter CI/CD

on:

push:

branches: [main, develop]

pull_request:

branches: [main]

jobs:

test:

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v3
- **uses:** subosito/flutter-action@v2
- with:**
 - flutter-version:** '3.19.0'
- **run:** flutter pub get
- **run:** flutter analyze
- **run:** flutter test

build_android:

needs: test

runs-on: ubuntu-latest

steps:

- **uses:** actions/checkout@v3
- **uses:** subosito/flutter-action@v2
- **run:** flutter build apk --release
- **uses:** actions/upload-artifact@v3
- with:**
 - name:** android-release
 - path:** build/app/outputs/flutter-apk/app-release.apk

build_ios:

needs: test

runs-on: macos-latest

steps:

- **uses:** actions/checkout@v3
- **uses:** subosito/flutter-action@v2
- **run:** flutter build ios --release --no-codesign

10.4 Performance Optimization

Image Optimization:

dart

```
CachedNetworkImage(  
  imageUrl: subscription.logoUrl,  
  memCacheWidth: 200, // Resize in memory  
  placeholder: (context, url) => Shimmer.fromColors(  
    baseColor: Colors.grey[300]!,  
    highlightColor: Colors.grey[100]!,  
    child: Container(width: 50, height: 50, color: Colors.white),  
  ),  
  errorWidget: (context, url, error) => Icon(Icons.error),  
)
```

List Performance:

dart

```
ListView.builder(  
  itemCount: subscriptions.length,  
  itemBuilder: (context, index) {  
    return SubscriptionCard(  
      key: ValueKey(subscriptions[index].id),  
      subscription: subscriptions[index],  
    );  
  },  
)
```

Build Optimization:

yaml

```
# pubspec.yaml  
flutter:  
  uses-material-design: true  
  
# Remove unused assets in production  
# Use vector graphics (SVG) where possible  
# Enable code shrinking in build.gradle
```

10.5 Monitoring & Analytics

Firestore Analytics Events: